Data Collection Strategy

Predicting High-ROI Supply Chain Entry Points via Cross-Domain Graph Analysis

To build a high-fidelity **Heterogeneous Graph** for predictive forecasting, data must be synthesized across the software registry (npm), the identity layer (Maintainers), and the infrastructure layer (GitHub/CI/CD). This document outlines the technical requirements and collection processes for the four foundational data phases.

Phase 1: Core Dependency Topology

The first phase establishes the "Skeleton" of the graph, mapping the interconnected dependencies within the npm ecosystem.

- **Source:** Google Open Source Insights (*deps.dev*) via Google BigQuery.
- **Access Point:** `bigquery-public-data.deps_dev_v1.Dependencies`
- **Data Points:** Package Name, Version, Release Timestamp, and Dependency Edge Lists (System, Name, Version, DependencyName).

Collection Process:

1. **GCP Configuration:** Enable the BigQuery API and query the pre-computed dependency tables. To ensure the graph is computationally manageable for a Graph Neural Network (GNN), filter for the **Top 100,000 to 250,000** packages based on dependent count.

2. **Temporal Isolation:** Use SQL filtering to extract the graph state as it existed on a specific historical date (e.g., August 1, 2025). This is critical for validating the model's predictive accuracy against subsequent real-world attacks.

Phase 2: The Maintainer & Ownership Layer

This layer facilitates the calculation of *Wormability*—measuring the potential lateral reach gained by compromising a specific developer identity.

- **Source:** npm Public Registry API / CouchDB Replica.
- **Data Points:** Maintainer Username, Email (hashed), and associated Package Ownership lists.

Collection Process:

1. **Metadata Extraction:** Iterate through the prioritized package list and fetch full JSON metadata from the npm registry (`https://registry.npmjs.org/[package]`).

2. **Bipartite Mapping:** Construct a maintainer-to-package edge list. Calculate the out-degree for each maintainer node to establish the "bridge" potential for self-propagating worms.

Phase 3: Infrastructure & CI/CD Evidence

This phase provides the cross-domain novelty by identifying nodes that bridge the software registry to privileged infrastructure environments.

- **Source:** GitHub GraphQL API (v4).
- **Data Points:** Presence of `.github/workflows/*.yml` files, repository configurations, and OIDC (Trusted Publishing) indicators.

Collection Process:

1. **URL Mapping:** Extract GitHub repository URLs from the npm `repository` metadata field. Clean and normalize these URLs into `owner/repo` formats.

2. **GraphQL Execution:** Execute bulk queries to verify the existence of the workflows directory. This approach avoids the strict rate limits of the REST API while allowing for deep inspection of infrastructure-as-code files.

Phase 4: Ground Truth & Historical Targets

Model training requires ground-truth labels of packages that were successfully exploited in high-impact campaigns.

- **Source:** OSV (Open Source Vulnerabilities) and GitHub Advisory Database.
- **Data Points:** Package Name, Version, Vulnerability Classification (Targeted Compromise vs. Logic Bug).

Collection Process:

1. **Incident Isolation:** Download the bulk npm advisory dump from *osv.dev*. Filter for incidents involving "Account Takeover," "Malicious Code Injection," or "Dependency Confusion" between Sept 2025 and April 2026.

2. **Node Labeling:** Match these targets against your node list to create the binary labels ($y=1$) for the GNN classification task.

Technical Summary Table

Component	Source	Primary Metric	Research Purpose
Dependencies	deps.dev	Transitive Reach	Blast Radius Foundation
Maintainers	npm API	Ownership Out-Degree	Wormability Metric
CI/CD Evidence	GitHub GraphQL	Workflow Presence	Infrastructure Exposure
Historical Hacks	OSV / Advisories	Compromise Labels	Training & Validation

> ***Implementation Note:*** *Given the Arch Linux environment, it is recommended to store these datasets in a local* ***PostgreSQL*** *instance. This enables high-speed*

> *joining of npm metadata with GitHub infrastructure flags, which is significantly more efficient than processing flat CSV files for large-scale graph construction.*